INF672 · MACHINE LEARNING · CUADERNO DE APUNTES

Regresión Logística

De la sigmoide a la clasificación multiclase: teoría, derivaciones, implementación y evaluación

$$\sigma(z) = 1 / (1 + e^{-z})$$

Universidad Autónoma Tomás Frías

Facultad de Ciencias Puras · Carrera de Ingeniería Informática

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

Gestión 2026-02 · Potosí, Bolivia

Objetivos de aprendizaje

Al terminar de estudiar este cuaderno, el estudiante será capaz de:

1. Explicar por qué la regresión lineal no es adecuada para problemas de clasificación.
2. Describir la función sigmoide, sus propiedades y su interpretación como probabilidad.
3. Interpretar el modelo en términos de *odds* y *log-odds* (logit).
4. Derivar la función de costo de entropía cruzada a partir del principio de máxima verosimilitud.
5. Calcular el gradiente e implementar el entrenamiento por descenso de gradiente desde cero.
6. Usar `LogisticRegression` de scikit-learn, interpretar sus coeficientes y aplicar regularización.
7. Evaluar un clasificador con matriz de confusión, precisión, *recall*, F1, ROC-AUC y curva PR, y elegir un umbral adecuado.
8. Extender el modelo a más de dos clases con One-vs-Rest y Softmax.

Prerrequisitos

Manejo de NumPy y Matplotlib (GL02), álgebra lineal con vectores y matrices (GL03), y derivadas con regla de la cadena (GL04). El cuaderno Jupyter que acompaña a estos apuntes (`INF672_Regresion_Logistica.ipynb`) contiene todo el código ejecutable; aquí se reproducen los fragmentos esenciales.

Contenido

01 Motivación: ¿por qué no usar regresión lineal?
02 La función sigmoide
03 El modelo: odds y logit
04 Frontera de decisión
05 Función de costo: entropía cruzada
06 Gradiente y descenso de gradiente
07 Implementación desde cero
08 scikit-learn e interpretación de coeficientes
09 Regularización
10 Preprocesamiento y trampas comunes
11 Evaluación del clasificador
12 Clasificación multiclase
13 Síntesis y cierre
14 Ejercicios propuestos

Convenciones de notación

Símbolo	Significado
m	Número de ejemplos (filas) del conjunto de entrenamiento
n	Número de características (columnas)
$\mathbf{x} = (x_1, \ldots, x_n)$	Vector de características de un ejemplo
X	Matriz de datos de tamaño m × n (un ejemplo por fila)
$y \in \{0, 1\}$	Etiqueta real; 1 = clase positiva
$x^{(i)}$, $y^{(i)}$	Características y etiqueta del i-ésimo ejemplo
$\mathbf{w}$, b	Vector de pesos y sesgo (intercepto): los parámetros a aprender
z	Logit o puntaje lineal: $z = \mathbf{w}^T\mathbf{x} + b$
$\hat{p}$	Probabilidad estimada de la clase positiva: $\hat{p} = \sigma(z)$
$\hat{y}$	Clase predicha (0 o 1) tras aplicar un umbral τ
$J(\mathbf{w}, b)$	Función de costo
α, λ	Tasa de aprendizaje y parámetro de regularización

01 Motivación: ¿por qué no usar regresión lineal?

Hasta ahora hemos trabajado con **regresión**: predecir un valor numérico continuo, como el precio de una vivienda o la temperatura de mañana. Muchos problemas reales, sin embargo, piden algo distinto: decidir a qué **categoría** pertenece un ejemplo. Esto se llama **clasificación**. Cuando solo hay dos categorías hablamos de **clasificación binaria**, y por convención codificamos la etiqueta como $y \in \{0, 1\}$, donde 1 representa la clase que nos interesa detectar (la clase *positiva*).

Problema	y = 1 (positivo)	y = 0 (negativo)
Rendimiento académico	El estudiante aprueba	El estudiante reprueba
Filtro de correo	Spam	Correo legítimo
Diagnóstico médico	Tumor maligno	Tumor benigno
Riesgo crediticio	El cliente incumple el pago	El cliente paga
Seguridad informática	Conexión maliciosa	Conexión normal

> NOTA · **Positivo no significa bueno**
>
> La palabra *positivo* no tiene connotación moral: es simplemente la clase que queremos detectar. En diagnóstico médico la clase positiva es *la enfermedad*; en seguridad, *el ataque*.

1.1 Un primer intento ingenuo

Como ya sabemos ajustar rectas, la primera idea natural es reutilizar la regresión lineal: ajustamos $\hat{y} = w \cdot x + b$ a los datos con etiquetas 0 y 1 y, para decidir, clasificamos como 1 todo ejemplo con $\hat{y} \geq 0.5$. Probemos esta idea con un caso sencillo: las horas que estudiaron 14 estudiantes y si aprobaron o no el examen. Luego añadimos tres estudiantes que estudiaron muchísimo (16, 18 y 20 horas) y, evidentemente, aprobaron.

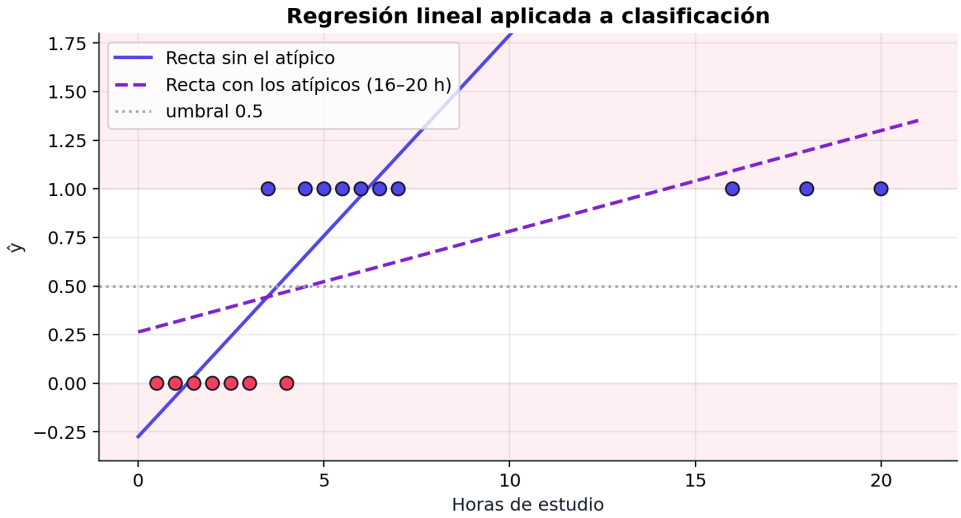


Figura 1. *Regresión lineal aplicada a clasificación. Las franjas coral marcan valores sin sentido como probabilidad (mayores que 1 o menores que 0).*

```
Out[0]:
Umbral sin atípicos: 3.75 h
Umbral con atípicos: 4.56 h   <- se desplazó por solo tres puntos
El estudiante que estudió 4.5 h y aprobó ahora se clasifica como: 0  (reprobado)
Predicción para 20 h (recta 1): 3.86   <- ¿una 'probabilidad' mayor que 1?
```

El experimento revela tres problemas de fondo:

1. **Salidas fuera de [0, 1].** La recta predice 3.86 para quien estudió 20 horas y valores negativos para quien casi no estudió. Ninguno de esos números puede interpretarse como una probabilidad, que es justamente lo que nos gustaría obtener.
2. **Sensibilidad a puntos extremos.** Los tres estudiantes nuevos ya estaban *bien clasificados*: cualquier modelo razonable diría que quien estudia 20 horas aprueba. Aun así, su presencia aplana la recta y mueve el umbral de 3.75 h a 4.56 h, de modo que un estudiante que antes se clasificaba bien (4.5 h, aprobó) ahora se clasifica mal. Datos que *confirman* el patrón empeoran el modelo: algo está mal planteado.
3. **El costo cuadrático castiga lo que no debe.** La regresión lineal minimiza el error cuadrático, así que penaliza fuertemente al punto de 20 h por estar lejos de la recta, aunque su clase sea evidente. El modelo se esfuerza en «acercarse» a ese punto en lugar de en separar bien las clases.

> IDEA CLAVE · **Lo que realmente necesitamos**
>
> Una función que tome cualquier número real (positivo, negativo, grande o pequeño) y lo transforme, de forma **suave** y **creciente**, en un valor dentro del intervalo (0, 1) que podamos leer como probabilidad. Además, que los ejemplos claramente bien clasificados no distorsionen el modelo. Esa función es la **sigmoide**, y el modelo resultante es la **regresión logística**.

02 La función sigmoide

La función **sigmoide** o **logística** se define como:

$$\sigma(z) = 1 \,/\, (1 + e^{-z}) \tag{2.1}$$

Para entender su comportamiento basta con analizar qué ocurre con e^{-z}:

- Si z es **muy grande y positivo**, $e^{-z} \approx 0$, de modo que $\sigma(z) \approx 1/(1 + 0) = 1$.
- Si z = **0**, $e^{0} = 1$, de modo que $\sigma(0) = 1/(1 + 1) = 0.5$.
- Si z es **muy grande y negativo**, e^{-z} es enorme, de modo que $\sigma(z) \approx 1/(\text{enorme}) \approx 0$.

Es decir, σ convierte la recta real completa en el intervalo abierto (0, 1), sin llegar nunca a tocar los extremos. Su gráfica tiene forma de «S» (de ahí su nombre: *sigma*).

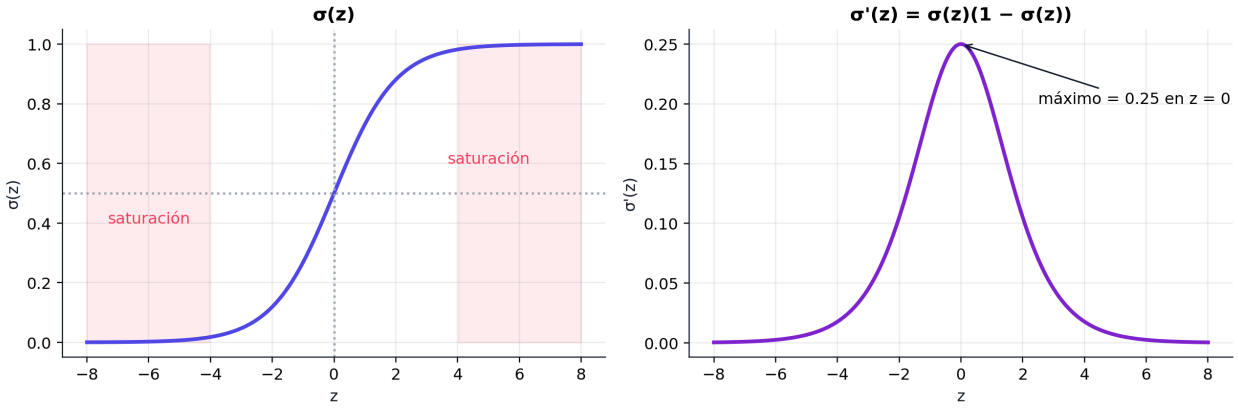


Figura 2. *Izquierda: la sigmoide y sus zonas de saturación. Derecha: su derivada, con máximo 0.25 en z = 0.*

2.1 Propiedades fundamentales

Propiedad	Expresión	Significado
Rango	$\sigma(z) \in (0, 1)$	Su salida se puede interpretar como probabilidad
Punto medio	$\sigma(0) = 0.5$	z = 0 es el punto de máxima incertidumbre
Monotonía	$z_1 < z_2 \Rightarrow \sigma(z_1) < \sigma(z_2)$	Mayor puntaje implica mayor probabilidad
Simetría	$\sigma(-z) = 1 - \sigma(z)$	La probabilidad de la clase 0 es la imagen espejo
Derivada	$\sigma'(z) = \sigma(z)\cdot(1 - \sigma(z))$	Se calcula reutilizando el valor de σ(z)
Inversa	$\sigma^{-1}(p) = \ln(p \,/\, (1 - p))$	La función logit (sección 3)

La **simetría** se comprueba directamente: $\sigma(-z) = 1/(1 + e^{z})$. Multiplicando numerador y denominador por e^{-z} se obtiene $e^{-z}/(e^{-z} + 1) = 1 - 1/(1 + e^{-z}) = 1 - \sigma(z)$.

2.2 Derivada de la sigmoide

La derivada de σ aparecerá al calcular el gradiente del modelo, así que conviene obtenerla con cuidado. Escribimos $\sigma(z) = (1 + e^{-z})^{-1}$ y aplicamos la regla de la cadena (derivada de u^{-1} es $-u^{-2}\cdot u'$):

$$\sigma'(z) = -(1+e^{-z})^{-2}\cdot(-e^{-z}) = e^{-z} / (1+e^{-z})^{2}$$
$$= [\,1/(1+e^{-z})\,]\cdot[\,e^{-z}/(1+e^{-z})\,] \tag{2.2}$$

El primer factor es σ(z). Para el segundo, sumamos y restamos 1 en el numerador:

$$e^{-z}/(1+e^{-z}) = [\,(1+e^{-z}) - 1\,]/(1+e^{-z}) = 1-\sigma(z)$$
$$\therefore\quad \sigma'(z) = \sigma(z)\cdot(1-\sigma(z)) \tag{2.3}$$

Esta expresión es muy cómoda: una vez calculada σ(z), su derivada sale casi gratis. Además nos dice que la derivada es máxima en z = 0, donde vale 0.5 × 0.5 = **0.25**, y se aproxima a cero en los extremos.

> ADVERTENCIA · **Saturación**
>
> Cuando |z| es grande (por ejemplo |z| > 5), σ′(z) ≈ 0: la sigmoide está **saturada** y es casi plana. Si el modelo estuviera muy seguro de una respuesta equivocada y el aprendizaje dependiera solo de σ′, apenas aprendería. Veremos en la sección 6 que la función de costo correcta (entropía cruzada) cancela este factor y elimina el problema; es uno de los argumentos fuertes contra el uso del error cuadrático.

2.3 Implementación numéricamente estable

La implementación directa `1 / (1 + np.exp(-z))` funciona en la mayoría de los casos, pero para z muy negativo (por ejemplo z = −1000) el término `np.exp(1000)` desborda a infinito y NumPy emite advertencias. La solución es usar dos fórmulas matemáticamente equivalentes según el signo de z: para $z \geq 0$ usamos $1/(1+e^{-z})$ y para $z < 0$ usamos $e^{z}/(1+e^{z})$, que nunca calculan la exponencial de un número grande y positivo.

In [1]:
```
def sigmoide(z):
    z = np.asarray(z, dtype=float)
    out = np.empty_like(z)
    pos = z >= 0
    out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))    # z >= 0: exp(-z) <= 1
    ez = np.exp(z[~pos])                        # z <  0: exp(z)  <  1
    out[~pos] = ez / (1.0 + ez)
    return out


def d_sigmoide(z):
    s = sigmoide(z)
    return s * (1 - s)


# Comprobación de la derivada con diferencias finitas centradas
h, z_test = 1e-5, np.array([-3.0, 0.0, 1.5, 6.0])
print("Analítica:", d_sigmoide(z_test))
print("Numérica :", (sigmoide(z_test + h) - sigmoide(z_test - h)) / (2 * h))
```

Out[1]:
```
Analítica: [0.0452 0.25   0.1491 0.0025]
Numérica : [0.0452 0.25   0.1491 0.0025]
```

Ambos cálculos coinciden, lo que confirma la fórmula (2.3). Observa también cómo la derivada en z = 6 es apenas 0.0025: la saturación en acción.

03 El modelo: odds y logit

3.1 Definición del modelo

La regresión logística combina dos ingredientes que ya conocemos. Primero, como en la regresión lineal, calcula una **combinación lineal** de las características. Segundo, pasa ese resultado por la sigmoide para convertirlo en probabilidad:

$$z = \mathbf{w}^{T}\mathbf{x} + b = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$$
$$\hat{p} = P(y = 1 \mid \mathbf{x}) = \sigma(z) = 1 \,/\, (1 + e\text{^}{-}(\mathbf{w}^{T}\mathbf{x} + b)) \qquad (3.1)$$

- **w** es el vector de **pesos**, uno por característica. Indica cuánto y en qué dirección influye cada variable.
- **b** es el **sesgo** o intercepto: desplaza el punto en que la probabilidad vale 0.5.
- **z** es el **logit** o puntaje: un número real cualquiera, positivo cuando el modelo se inclina por la clase 1.
- $\hat{\mathbf{p}}$ es la probabilidad estimada de la clase positiva. La de la clase negativa es, simplemente, 1 − $\hat{p}$.

Para todos los ejemplos a la vez, en forma matricial, escribimos $\hat{\mathbf{p}} = \sigma(X\mathbf{w} + b)$, donde σ se aplica elemento a elemento. Esta forma vectorizada es la que usaremos en el código, porque evita bucles y es mucho más rápida.

El modelo entrega una probabilidad; para obtener una **clase** necesitamos un **umbral** de decisión τ, que por defecto es 0.5:

$$\hat{y} = 1 \ \text{si} \ \hat{p} \geq \tau\,; \qquad \hat{y} = 0 \ \text{si} \ \hat{p} < \tau \qquad (3.2)$$

EJEMPLO · Cálculo a mano

Supongamos un modelo con dos características ya entrenado: $w_1 = 0.8$, $w_2 = -0.5$, $b = -1$. Para un ejemplo $x = (3, 1)$:

$z = 0.8 \cdot 3 + (-0.5) \cdot 1 + (-1) = 2.4 - 0.5 - 1 = 0.9$, $\hat{p} = \sigma(0.9) = 1/(1 + e^{-0.9}) \approx 1/(1 + 0.407) \approx$ **0.711**.

Con $\tau = 0.5$ la predicción es $\hat{y} = 1$. El modelo estima un 71.1 % de probabilidad de que el ejemplo sea positivo.

3.2 Odds y log-odds: ¿por qué se llama «regresión»?

Resulta extraño que un método de clasificación se llame *regresión*. La explicación está en un concepto muy usado en estadística y en apuestas: los **odds** (en español, *momios* o *chances*). Los odds comparan la probabilidad de que algo ocurra con la de que no ocurra:

$$\text{odds} = p \,/\, (1 - p) \tag{3.3}$$

Si p = 0.8, los odds son 0.8/0.2 = 4: decimos que el evento está «4 a 1 a favor». Si p = 0.5, los odds son 1 («uno a uno»). Ahora despejamos z de la ecuación p = σ(z):

$$p = 1 \,/\, (1 + e^{-z}) \;\Rightarrow\; 1/p = 1 + e^{-z} \;\Rightarrow\; e^{-z} = (1 - p)/p$$
$$\Rightarrow\; z = \ln(\, p \,/\, (1 - p)\,) = \mathbf{w}^{T}\mathbf{x} + b \tag{3.4}$$

> IDEA CLAVE · **El logit es lineal**
>
> El **logaritmo de los odds** —llamado **logit**— es una función **lineal** de las características. La regresión logística es, literalmente, una regresión lineal sobre el logit. La sigmoide es la función inversa del logit: lo que hace es traducir ese puntaje lineal de vuelta al mundo de las probabilidades.

p	odds = p/(1−p)	logit = ln(odds)	σ(logit)
0.01	0.010	−4.595	0.01
0.10	0.111	−2.197	0.10
0.25	0.333	−1.099	0.25
0.50	1.000	0.000	0.50
0.75	3.000	1.099	0.75
0.90	9.000	2.197	0.90
0.99	99.000	4.595	0.99

La tabla muestra la cadena de transformaciones: la probabilidad vive en (0, 1), los odds en (0, ∞) y el logit en (−∞, ∞), que es justamente el rango de una expresión lineal. El logit es simétrico (p = 0.25 y p = 0.75 dan ∓1.099) y la última columna confirma que σ deshace el logit.

04 Frontera de decisión

Con el umbral $\tau = 0.5$, el modelo predice la clase 1 cuando $\sigma(z) \geq 0.5$. Como σ es creciente y $\sigma(0) = 0.5$, esto ocurre exactamente cuando $z \geq 0$. Por tanto, la **frontera de decisión** —el lugar donde el modelo «cambia de opinión»— es el conjunto de puntos que cumplen:

$$\mathbf{w}^{\mathrm{T}}\mathbf{x} + b = 0 \tag{4.1}$$

En dos dimensiones esta ecuación es una **recta**: $w_1x_1 + w_2x_2 + b = 0$, que podemos despejar como $x_2 = -(w_1x_1 + b)/w_2$. En tres dimensiones es un plano, y en n dimensiones un **hiperplano**. Por eso se dice que la regresión logística es un **clasificador lineal**: separa las clases con una frontera plana.

4.1 Qué controla cada parámetro

- La **dirección** de **w** es perpendicular a la frontera y apunta hacia la región de la clase 1.
- La **magnitud** $\|\mathbf{w}\|$ controla qué tan abrupta es la transición: con pesos grandes, la probabilidad pasa de casi 0 a casi 1 en una franja estrecha (modelo «muy seguro»); con pesos pequeños, la transición es gradual.
- El **sesgo** b desplaza la frontera en paralelo, sin rotarla.

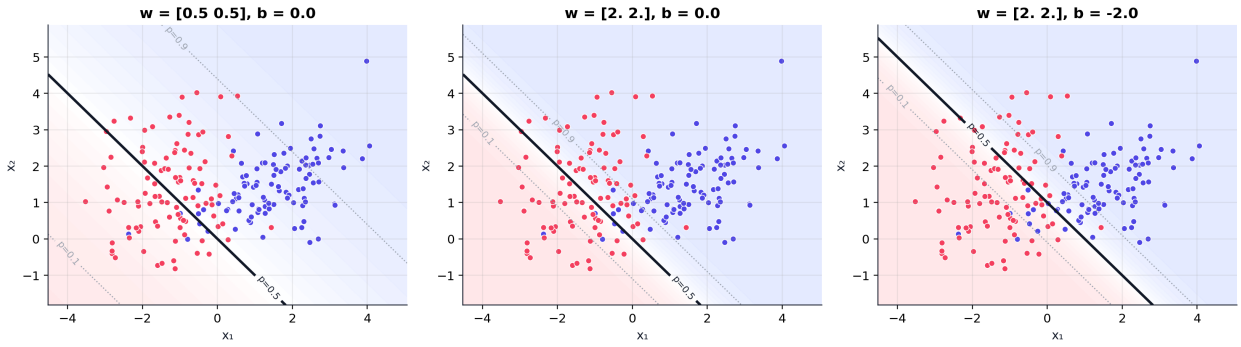


Figura 3. *Fronteras con pesos fijados a mano. Paneles 1 y 2: misma dirección de w, misma frontera, pero distinta pendiente de la probabilidad (líneas punteadas: p = 0.1 y p = 0.9). Panel 3: cambiar b traslada la frontera.*

4.2 Fronteras no lineales con características polinomiales

¿Qué ocurre cuando las clases no pueden separarse con una recta, como en datos con forma de anillos concéntricos? El modelo sigue siendo lineal **en los parámetros**, pero nada impide darle **características nuevas** construidas a partir de las originales, como x_1^2, x_2^2 y x_1x_2. Con ellas el logit queda, por ejemplo:

$$z = w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2 + b \tag{4.2}$$

y la ecuación z = 0 ya puede describir una circunferencia o una elipse en el espacio original.

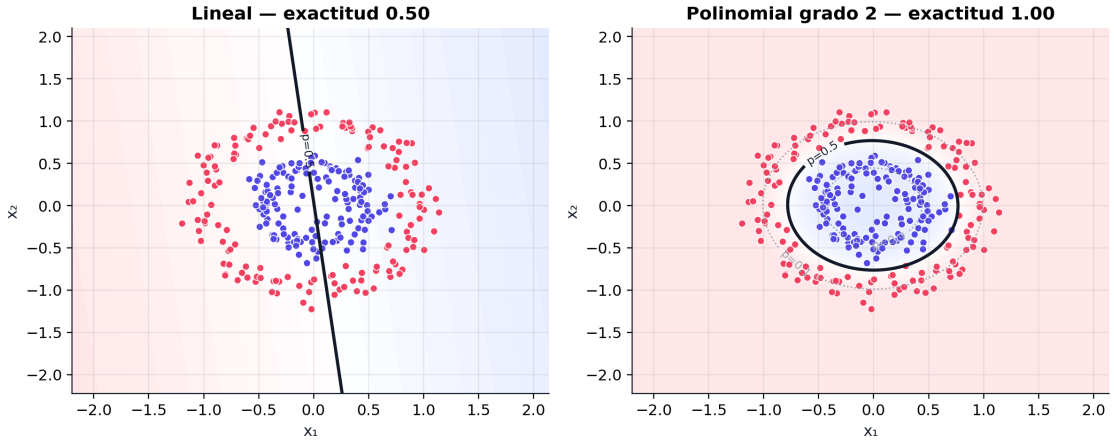


Figura 4. *Datos en anillos. El modelo lineal no puede separarlos (exactitud 0.50); con características de grado 2 la frontera se vuelve circular (exactitud 1.00).*

```
Out[1]:
Coeficientes aprendidos:
      x1: -0.083
      x2: +0.019
   x1^2: -5.493
  x1 x2: -0.119
   x2^2: -5.616
```

Los coeficientes de x_1^2 y x_2^2 son grandes, negativos y casi iguales, mientras que los demás son prácticamente nulos. El modelo aprendió algo de la forma $-5.5 \cdot (x_1^2 + x_2^2) + b = 0$: **una circunferencia**. La regresión logística no «sabe» de círculos; somos nosotros quienes le damos las características adecuadas. A esto se le llama **ingeniería de características**.

> **ADVERTENCIA**
>
> Con grados altos el número de características crece muy rápido (con 10 variables y grado 3 ya hay 285) y el modelo puede **sobreajustar**: memorizar el ruido del entrenamiento. La herramienta para controlar esto es la regularización (sección 9).

05 Función de costo: entropía cruzada

Ya tenemos el modelo; falta decidir cómo medir qué tan buenos son unos parámetros (**w**, b) para luego buscar los mejores. Esa medida es la **función de costo** J(**w**, b). Primero veremos por qué el error cuadrático no sirve y luego derivaremos la función correcta desde un principio estadístico.

5.1 ¿Por qué no el error cuadrático medio?

$$J(\mathbf{w}, b) = (1/m) \cdot \sum_i \left(\sigma(z^{(i)}) - y^{(i)} \right)^2 \qquad [\text{MSE}] \tag{5.1}$$

Tiene dos defectos importantes cuando se combina con la sigmoide:

1. **No es convexa.** Al componer el cuadrado con la sigmoide, la superficie de costo adquiere regiones casi planas y forma de «S». El descenso de gradiente puede quedar atrapado o avanzar con enorme lentitud.
2. **La penalización está acotada.** Un ejemplo con y = 1 al que el modelo asigna $\hat{p} = 0.001$ —un error gravísimo, con total seguridad— cuesta como máximo $(0.001 - 1)^2 \approx 1$. El modelo no recibe una señal fuerte para corregir sus errores más graves.

5.2 Derivación desde la máxima verosimilitud

En lugar de inventar una función de costo, la deducimos de un principio general: elegir los parámetros que hacen **más probable** haber observado los datos que realmente tenemos. Este es el **principio de máxima verosimilitud**.

Paso 1: modelo probabilístico de una etiqueta. Cada etiqueta y es una variable de Bernoulli: vale 1 con probabilidad $\hat{p}$ y 0 con probabilidad $1 - \hat{p}$. Ambos casos se pueden escribir en una sola expresión:

$$P(y \mid \mathbf{x}) = \hat{p}^{y} \cdot (1 - \hat{p})^{1-y} \tag{5.2}$$

Compruébalo: si y = 1 la expresión vale $\hat{p}^1 \cdot (1 - \hat{p})^0 = \hat{p}$; si y = 0 vale $\hat{p}^0 \cdot (1 - \hat{p})^1 = 1 - \hat{p}$.

Paso 2: verosimilitud de todo el conjunto. Suponiendo que los ejemplos son independientes, la probabilidad de observar todas las etiquetas es el producto de las probabilidades individuales:

$$L(\mathbf{w}, b) = \prod_i (\hat{p}^{(i)})^{y^{(i)}} \cdot (1 - \hat{p}^{(i)})^{(1 - y^{(i)})} \tag{5.3}$$

Paso 3: tomar logaritmo. El producto de cientos de probabilidades es un número diminuto que la computadora no puede representar bien. Como el logaritmo es creciente, maximizar L es lo mismo que maximizar ln L, y el logaritmo convierte productos en sumas y exponentes en factores:

$$\ell(\mathbf{w}, b) = \sum_i \left[y^{(i)} \cdot \ln \hat{p}^{(i)} + (1 - y^{(i)}) \cdot \ln(1 - \hat{p}^{(i)}) \right] \tag{5.4}$$

Paso 4: convertir en costo. Los algoritmos de optimización suelen *minimizar*. Maximizar ℓ equivale a minimizar $-\ell$; si además promediamos sobre los m ejemplos, obtenemos la **entropía cruzada binaria**, también llamada **log-loss**:

$$J(\mathbf{w}, b) = -(1/m) \cdot \sum_i \left[y^{(i)} \cdot \ln \hat{p}^{(i)} + (1 - y^{(i)}) \cdot \ln(1 - \hat{p}^{(i)}) \right] \tag{5.5}$$

5.3 Intuición: analizando cada caso

Para un solo ejemplo, solo uno de los dos términos está «activo»:

Etiqueta real	Costo del ejemplo	Si acierta con seguridad	Si falla con seguridad
y = 1	$-\ln \hat{p}$	$\hat{p} \to 1 \Rightarrow$ costo $\to 0$	$\hat{p} \to 0 \Rightarrow$ costo $\to \infty$
y = 0	$-\ln(1 - \hat{p})$	$\hat{p} \to 0 \Rightarrow$ costo $\to 0$	$\hat{p} \to 1 \Rightarrow$ costo $\to \infty$

Estar **seguro y equivocado** se castiga sin límite, mientras que acertar con seguridad casi no cuesta nada. Un ejemplo con y = 1 y $\hat{p}$ = 0.9 cuesta $-\ln 0.9 \approx 0.105$; con $\hat{p}$ = 0.5 cuesta 0.693; con $\hat{p}$ = 0.01 cuesta 4.6.

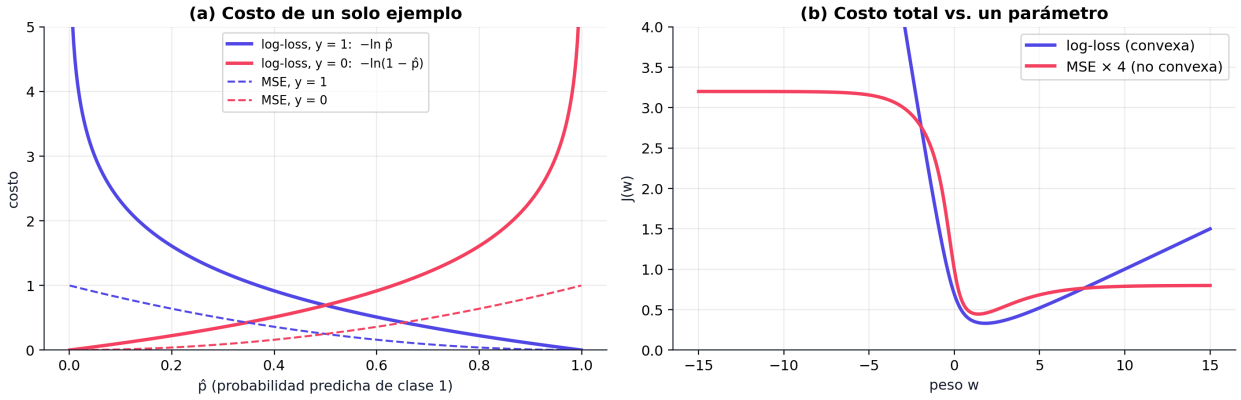


Figura 5. *(a) Costo de un ejemplo: la log-loss crece sin límite ante errores seguros; el MSE nunca supera 1. (b) Costo total frente a un único peso: la log-loss es convexa; el MSE tiene mesetas.*

DEFINICIÓN · Convexidad

Una función es **convexa** si el segmento que une dos puntos cualesquiera de su gráfica queda siempre por encima (o sobre) ella; intuitivamente, tiene forma de «taza». En una función convexa **todo mínimo local es también global**, de modo que el descenso de gradiente, con una tasa de aprendizaje adecuada, encuentra la mejor solución posible sin quedar atrapado. La log-loss de la regresión logística es convexa respecto de (**w**, b).

06 Gradiente y descenso de gradiente

6.1 Derivación del gradiente

Para minimizar J necesitamos su gradiente: el vector de derivadas parciales respecto de cada parámetro. Trabajamos primero con **un solo ejemplo**, cuyo costo es J = −[y·ln p̂ + (1 − y)·ln(1 − p̂)], con p̂ = σ(z) y z = $\mathbf{w}^T\mathbf{x}$ + b. La dependencia es una cadena: $w_j \to z \to \hat{p} \to J$, así que aplicamos la regla de la cadena:

$$\partial J/\partial w_j = (\partial J/\partial \hat{p}) \cdot (\partial \hat{p}/\partial z) \cdot (\partial z/\partial w_j) \tag{6.1}$$

Calculamos cada factor por separado:

$$\text{①}\ \partial J/\partial \hat{p} = -y/\hat{p} + (1-y)/(1-\hat{p}) = (\hat{p}-y)\ /\ [\,\hat{p}\cdot(1-\hat{p})\,]$$
$$\text{②}\ \partial \hat{p}/\partial z = \sigma'(z) = \hat{p}\cdot(1-\hat{p}) \qquad \text{(ecuación 2.3)}$$
$$\text{③}\ \partial z/\partial w_j = x_j\,, \qquad \partial z/\partial b = 1$$

El paso ① requiere sumar las dos fracciones con denominador común: [−y(1 − p̂) + (1 − y)p̂] / [p̂(1 − p̂)]; al desarrollar el numerador, −y + y·p̂ + p̂ − y·p̂ = p̂ − y. Al multiplicar los tres factores ocurre algo notable: **el término p̂·(1 − p̂) se cancela**.

$$\partial J/\partial w_j = (\hat{p}-y)\cdot x_j$$
$$\partial J/\partial b = \hat{p}-y \tag{6.2}$$

> IDEA CLAVE · **Por qué esta cancelación es tan importante**
>
> El gradiente resulta proporcional al **error** (p̂ − y), no a σ′(z). Aunque la sigmoide esté saturada, si el modelo se equivoca mucho, el error es grande y el gradiente también: el modelo aprende rápido justamente de sus peores errores. Con el MSE, en cambio, el factor σ′(z) no se cancela y el aprendizaje se frena en las zonas saturadas.

Promediando sobre los m ejemplos y escribiéndolo en forma matricial:

$$\partial J/\partial \mathbf{w} = (1/m)\cdot X^T(\hat{\mathbf{p}}-\mathbf{y})$$
$$\partial J/\partial b = (1/m)\cdot \textstyle\sum_i (\hat{p}^{(i)}-y^{(i)}) \tag{6.3}$$

> NOTA · **Coincidencia con la regresión lineal**
>
> Es **la misma fórmula** del gradiente del MSE en regresión lineal, $(1/m)\cdot X^{\intercal}(\hat{y} - y)$. La diferencia está solo en cómo se calcula la predicción: allí $\hat{y} = X\mathbf{w} + b$, aquí $\hat{p} = \sigma(X\mathbf{w} + b)$. No es casualidad: ambos modelos pertenecen a la familia de los **modelos lineales generalizados** y cada uno usa su función de enlace natural (identidad y logit, respectivamente).

6.2 Descenso de gradiente

El gradiente apunta en la dirección en que el costo **aumenta** más rápido. Para reducirlo, damos un pequeño paso en la dirección contraria y repetimos hasta que el costo deje de bajar. El tamaño del paso lo controla la **tasa de aprendizaje** α:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \partial J/\partial \mathbf{w}$$
$$b \leftarrow b - \alpha \cdot \partial J/\partial b \tag{6.4}$$

Existen tres variantes según cuántos ejemplos se usan para calcular el gradiente en cada actualización:

Variante	Ejemplos por actualización	Características
Batch (por lotes completo)	Todos (m)	Trayectoria suave y estable; cada paso es costoso con datasets grandes
Estocástico (SGD)	1	Muchísimas actualizaciones baratas; trayectoria ruidosa
Mini-batch	k (p. ej. 32 o 64)	Equilibrio entre ambos; es la opción usada en la práctica

> DEFINICIÓN · **Época**
>
> Una **época** es una pasada completa por todo el conjunto de entrenamiento. Con 426 ejemplos y mini-batches de 32, cada época contiene [426/32] = 14 actualizaciones de los parámetros.

6.3 Verificación del gradiente

Cuando implementamos un gradiente a mano es fácil equivocarse en un signo o en un índice, y el error no siempre es evidente: el modelo puede entrenar «más o menos» con un gradiente incorrecto. La técnica de **gradient checking** compara el gradiente analítico con una aproximación numérica por diferencias finitas, $[J(w_j + h) - J(w_j - h)] / 2h$, con h muy pequeño.

In [2]:
```
def costo_logloss(X, y, w, b, eps=1e-12):
    p = np.clip(sigmoide(X @ w + b), eps, 1 - eps)   # evita log(0)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))


def gradientes(X, y, w, b):
    error = sigmoide(X @ w + b) - y                  # (p̂ - y)
    return X.T @ error / X.shape[0], error.mean()     # ecuación (6.3)

# ... (cálculo de dw_num por diferencias finitas, ver notebook)
print("dw analítico:", dw)
print("dw numérico :", dw_num)
```

Out[2]:
```
dw analítico: [ 0.0049 -0.0465  0.0189  0.3321]
dw numérico : [ 0.0049 -0.0465  0.0189  0.3321]
db analítico: 0.042774 | db numérico: 0.042774
Diferencia relativa: 7.73e-09  -> ✓ correcto
```

Una diferencia relativa del orden de 10^{-7} o menor indica que el gradiente es correcto; valores del orden de 10^{-2} o mayores delatan un error.

07 Implementación desde cero

7.1 El dataset: Breast Cancer Wisconsin

Para trabajar con datos reales usamos un dataset clásico incluido en scikit-learn. Contiene **569 tumores** de mama, cada uno descrito por **30 características numéricas** calculadas a partir de imágenes digitalizadas de biopsias: radio, textura, perímetro, área, suavidad, concavidad, entre otras. Para cada medida se registran tres versiones: la media (*mean*), el error estándar (*error*) y el peor valor (*worst*).

La etiqueta original codifica 0 = maligno y 1 = benigno. **La invertimos** para que la clase positiva sea la que queremos detectar, es decir, 1 = maligno. De los 569 tumores, 212 (37.3 %) son malignos.

Característica	Media	Desv. est.	Mínimo	Máximo
mean radius	14.127	3.524	6.981	28.110
mean texture	19.290	4.301	9.710	39.280
mean area	654.889	351.914	143.500	2501.000
mean fractal dimension	0.063	0.007	0.050	0.097
worst smoothness	0.132	0.023	0.071	0.223

Las escalas son muy distintas: el área va de 143 a 2501, mientras que la dimensión fractal ronda 0.06. Esto tendrá consecuencias importantes en el entrenamiento, que estudiaremos en la sección 10. Por ahora, **estandarizamos** cada característica para que tenga media 0 y desviación estándar 1.

7.2 División y estandarización sin fuga de datos

```
In [3]:
X_train, X_test, y_train, y_test = train_test_split(
    X_bc, y_bc, test_size=0.25, stratify=y_bc, random_state=42)

escalador = StandardScaler()
X_train_s = escalador.fit_transform(X_train)   # aprende media y desviación SOLO del train
X_test_s  = escalador.transform(X_test)        # aplica esas mismas estadísticas al test
```

ADVERTENCIA · **Fuga de datos (data leakage)**

El escalador se **ajusta solo con el conjunto de entrenamiento** (`fit_transform`) y luego se **aplica** al de prueba (`transform`). Si calculáramos la media y la desviación con todo el dataset, el modelo «vería» información del conjunto de prueba y la evaluación saldría optimista. El conjunto de prueba debe simular datos que el modelo nunca ha visto.

El parámetro `stratify=y_bc` garantiza que ambos conjuntos conserven la proporción original de malignos (≈ 37 %).

7.3 La clase RegresionLogisticaGD

Encapsulamos todo lo anterior en una clase con la misma interfaz que scikit-learn (`fit`, `predict_proba`, `predict`, `score`). Soporta descenso por batch completo o mini-batch y regularización L2 opcional mediante el parámetro `lam` (que explicaremos en la sección 9).

In [4]:
```
class RegresionLogisticaGD:
    def __init__(self, alpha=0.1, n_iter=1000, lam=0.0, batch_size=None, tol=1e-7, seed=0):
        self.alpha, self.n_iter, self.lam = alpha, n_iter, lam
        self.batch_size, self.tol, self.seed = batch_size, tol, seed


    def fit(self, X, y):
        m, n = X.shape
        rng = np.random.default_rng(self.seed)
        self.w, self.b = np.zeros(n), 0.0                # 1. inicializar en cero
        self.historial = [self._costo(X, y)]
        bs = self.batch_size or m
        for _ in range(self.n_iter):                       # 2. por cada época
            idx = rng.permutation(m)                       #    barajar los ejemplos
            for ini in range(0, m, bs):                    # 3. por cada lote
                lote = idx[ini:ini + bs]
                dw, db = gradientes(X[lote], y[lote], self.w, self.b)
                dw += (self.lam / m) * self.w              #    término L2
                self.w -= self.alpha * dw                  # 4. actualizar
                self.b -= self.alpha * db
            self.historial.append(self._costo(X, y))
            if abs(self.historial[-2] - self.historial[-1]) < self.tol:
                break                                      # 5. criterio de parada
        return self


    def predict_proba(self, X):
        return sigmoide(X @ self.w + self.b)


    def predict(self, X, umbral=0.5):
        return (self.predict_proba(X) >= umbral).astype(int)
```

Algunas decisiones de diseño merecen comentario. Los pesos se inicializan en **cero**, lo cual es válido aquí porque la función de costo es convexa (en redes neuronales, en cambio, sería un error). En cada época se **barajan** los ejemplos para que los mini-batches sean distintos y no se introduzcan sesgos por el orden de los datos. Finalmente, el entrenamiento se detiene cuando el costo deja de mejorar más que una tolerancia `tol`.

In [5]:
```
modelo = RegresionLogisticaGD(alpha=0.1, n_iter=5000).fit(X_train_s, y_train)
print(f"Exactitud train: {modelo.score(X_train_s, y_train):.4f} | test: {modelo.score(X_test_s,
y_test):.4f}")
```

Out[5]:
```
Costo final (train): 0.0455
Exactitud train: 0.9906 | test: 0.9650
```

Con apenas unas líneas de NumPy el modelo clasifica correctamente el **96.5 %** de los tumores que nunca vio durante el entrenamiento.

7.4 Curvas de aprendizaje

La gráfica del costo frente a las iteraciones es la principal herramienta de diagnóstico del entrenamiento. La Figura 6 compara varias tasas de aprendizaje y las tres variantes del descenso de gradiente.

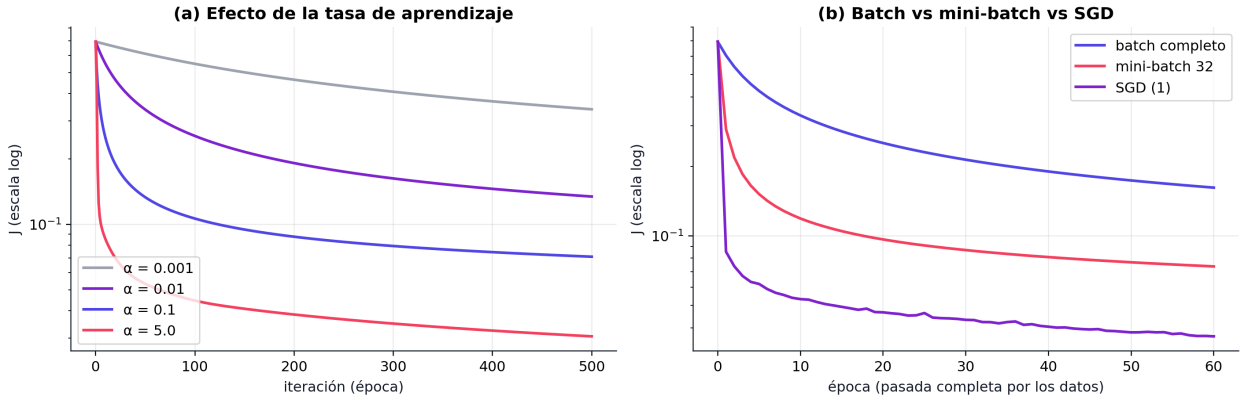


Figura 6. *(a) Efecto de la tasa de aprendizaje α. (b) Batch completo frente a mini-batch y SGD, medidos por época. Escala logarítmica en el eje vertical.*

- **Panel (a).** Con α = 0.001 el costo apenas baja: el aprendizaje es demasiado lento. Con α = 0.01 y 0.1 desciende de forma sostenida. Con α = 5 converge incluso más rápido: con datos estandarizados la log-loss es una superficie suave que tolera pasos grandes. Pero esto **no es lo habitual**: en otros problemas, y sobre todo en redes neuronales, un α excesivo hace que los pasos «salten» por encima del mínimo y el costo oscile o diverja.
- **Panel (b).** Medidos por época, los mini-batches realizan muchas más actualizaciones que el batch completo, por lo que reducen el costo más rápido. El SGD puro es el más rápido al inicio, pero también el más ruidoso.

> NOTA · **Regla práctica**
>
> Durante el entrenamiento el costo **siempre** debería disminuir. Si sube o rebota, reduce α. Si baja con exasperante lentitud, auméntalo. Una estrategia habitual es probar valores en escala logarítmica: 0.001, 0.01, 0.1, 1.

08 scikit-learn e interpretación de coeficientes

8.1 Comparación con nuestra implementación

La clase `LogisticRegression` de scikit-learn usa por defecto el optimizador **L-BFGS**, un método cuasi-Newton que aprovecha información de curvatura y converge en muchas menos iteraciones que el descenso de gradiente simple. Además, aplica **regularización L2 por defecto** con C = 1. Para compararla con nuestro modelo sin regularización usamos un valor de C enorme.

> NOTA · **Cambios de API en scikit-learn 1.8**
>
> Desde la versión 1.8 el parámetro `penalty` está obsoleto. La regularización se controla ahora con `C` (intensidad) y `l1_ratio` (0 = L2, 1 = L1, valores intermedios = Elastic Net). En tutoriales y libros anteriores encontrarás código como `LogisticRegression(penalty=None)` o `penalty="l1"`; funciona en versiones antiguas, pero emite advertencias en las nuevas. Asimismo, el parámetro `multi_class` fue eliminado en la versión 1.7.

In [6]:
```
sk = LogisticRegression(C=1e10, max_iter=10_000).fit(X_train_s, y_train)   # ≈ sin regularizar
gd = RegresionLogisticaGD(alpha=0.5, n_iter=200_000, tol=1e-12).fit(X_train_s, y_train)
```

Out[6]:
```
                     Desde cero      sklearn
log-loss train          0.00653      0.00013
exactitud test           0.9301       0.9371
          ||w||           48.84       620.54
```

Ambas implementaciones logran un costo de entrenamiento casi nulo, pero los resultados esconden dos señales de alerta:

- **La norma de los pesos es enorme** y distinta entre ambos métodos. El dataset es casi **linealmente separable**: sin regularización, el modelo puede seguir reduciendo el costo agrandando los pesos indefinidamente, volviendo la sigmoide cada vez más empinada (sección 10.3).
- **La exactitud de prueba empeora** respecto del modelo de la sección 7, que se detuvo antes (93 % frente a 96.5 %). El modelo memoriza el entrenamiento y generaliza peor: es **sobreajuste**.

Por eso, en la práctica, usamos la configuración por defecto, que ya incluye regularización:

In [7]:
```
clf = LogisticRegression(max_iter=1000).fit(X_train_s, y_train)      # C = 1, L2
print(f"Exactitud test: {clf.score(X_test_s, y_test):.4f}")
```

Out[7]:
```
Exactitud test: 0.9650
```

8.2 Interpretación de los coeficientes: odds ratio

Una de las grandes ventajas de la regresión logística es que sus coeficientes tienen un significado preciso. Partimos de la ecuación (3.4): ln(odds) = $\mathbf{w}^{\mathsf{T}}\mathbf{x}$ + b. Si aumentamos una característica x_j en **una unidad** dejando las demás fijas, el logit aumenta exactamente w_j. Al exponenciar:

$$\text{odds}(x_j + 1) \,/\, \text{odds}(x_j) = e\text{^}(w_j) \tag{8.1}$$

La cantidad $e\text{^}(w_j)$ se llama **odds ratio** (razón de momios) y nos dice por cuánto se **multiplican** los odds de la clase positiva cuando x_j aumenta en una unidad.

Coeficiente w_j	Odds ratio $e\text{^}(w_j)$	Interpretación
$w_j > 0$	> 1	Al aumentar x_j, aumentan los odds de la clase 1
$w_j = 0$	$= 1$	x_j no influye en la predicción
$w_i < 0$	< 1	Al aumentar x_i, disminuyen los odds de la clase 1

EJEMPLO · Cómo leer un coeficiente

Si una característica como `worst texture` tuviera w = 1.1, su odds ratio sería $e\text{^}1.1 \approx 3.0$. Como las características están estandarizadas, la «unidad» es **una desviación estándar**. Se leería así: *«un aumento de una desviación estándar en la textura máxima triplica los odds de que el tumor sea maligno, manteniendo constantes las demás variables»*.

Un coeficiente w = −0.69 daría $e\text{^}-0.69 \approx 0.5$: los odds se reducen a la mitad.

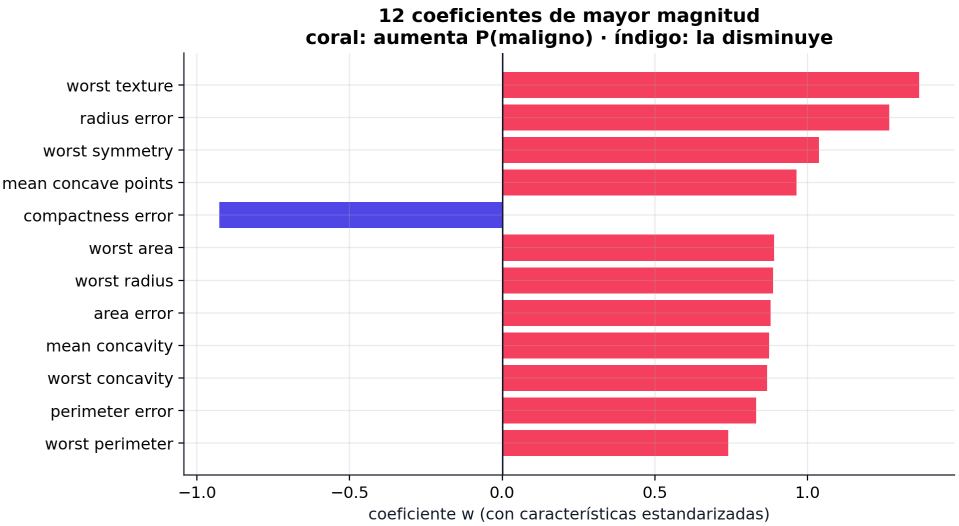


Figura 7. *Los 12 coeficientes de mayor magnitud del modelo con C = 1. En coral, las características que aumentan la probabilidad de malignidad; en índigo, las que la disminuyen.*

Las características con mayor peso positivo —textura, error del radio, simetría y puntos cóncavos en su peor valor— son coherentes con la intuición clínica: los tumores malignos tienden a ser más grandes, irregulares y con contornos más cóncavos.

ADVERTENCIA · **Cuidados al interpretar**

- El efecto es sobre los **odds**, no sobre la probabilidad. El cambio en p depende de dónde estemos en la sigmoide: es grande cerca de p = 0.5 y pequeño en las zonas saturadas.
- Solo con características **estandarizadas** pueden compararse las magnitudes entre variables. Sin estandarizar, la unidad es la original (mm, mm^2…).
- Con **multicolinealidad** los coeficientes individuales se vuelven inestables. En este dataset `mean radius`, `mean perimeter` y `mean area` tienen correlaciones superiores a 0.98: el modelo puede repartir el efecto entre ellas de forma arbitraria, e incluso asignar signos contraintuitivos. La predicción puede ser buena aunque los coeficientes no sean interpretables uno por uno.
- Asociación **no es causalidad**: un coeficiente grande indica que la variable ayuda a predecir, no que la cause.

09 Regularización

9.1 La idea

Vimos que sin restricciones el modelo puede agrandar sus pesos para ajustarse cada vez más al entrenamiento, y que eso empeora la generalización. La **regularización** añade a la función de costo un término que **penaliza los pesos grandes**, de modo que el optimizador debe equilibrar dos objetivos: ajustarse a los datos y mantener los pesos pequeños. El resultado son modelos más simples, más estables y que generalizan mejor.

Tipo	Término añadido a J	Efecto	scikit-learn ≥ 1.8
L2 (Ridge)	$(\lambda/2m)\cdot\sum_j w_j^2$	Encoge todos los pesos hacia 0 sin anularlos	`l1_ratio=0` (defecto)
L1 (Lasso)	$(\lambda/m)\cdot\sum_j \lvert w_j\rvert$	Anula pesos: selección de características	`l1_ratio=1`, solver `liblinear` o `saga`
Elastic Net	combinación de ambos	Selección + estabilidad con variables correlacionadas	`0 < l1_ratio < 1`, solver `saga`

Con L2, el costo y su gradiente quedan:

$$J(\mathbf{w}, b) = -(1/m)\cdot\sum_i \left[\, y^{(i)}\cdot \ln \hat{p}^{(i)} + (1 - y^{(i)})\cdot \ln(1 - \hat{p}^{(i)}) \,\right] + (\lambda/2m)\cdot\|\mathbf{w}\|^2$$
$$\partial J/\partial \mathbf{w} = (1/m)\cdot X^{T}(\hat{\mathbf{p}} - \mathbf{y}) + (\lambda/m)\cdot\mathbf{w} \qquad (9.1)$$

El término extra del gradiente, $(\lambda/m)\cdot\mathbf{w}$, empuja cada peso hacia cero en cada paso; por eso a la L2 también se le llama *weight decay* (decaimiento de pesos). Es exactamente lo que implementamos con el parámetro `lam` de nuestra clase. Observa que el sesgo b **no** se regulariza: solo desplaza la frontera y penalizarlo no aporta simplicidad.

> ADVERTENCIA · **El parámetro C funciona al revés**
>
> scikit-learn no usa λ sino **C = 1/λ**. Por tanto, **C pequeño significa regularización fuerte** y C grande, regularización débil. Es una fuente frecuente de confusión: «subir C» relaja la regularización.

9.2 Caminos de regularización

Para ver el efecto de la regularización entrenamos el modelo con 30 valores de C entre 0.001 y 100 y graficamos cómo cambia cada uno de los 30 coeficientes.

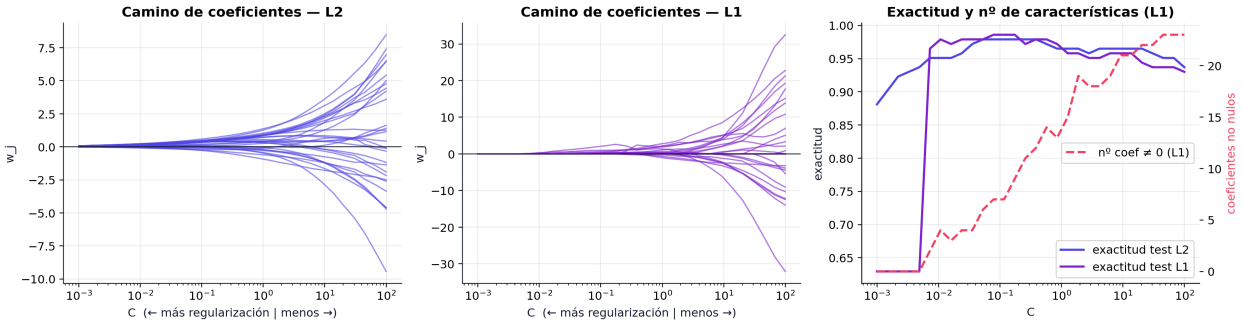


Figura 8. *Izquierda y centro: evolución de los 30 coeficientes con L2 y L1 según C. Derecha: exactitud de prueba y número de coeficientes no nulos (L1).*

- **Con L2**, al disminuir C todos los coeficientes se encogen suave y simultáneamente hacia cero, pero ninguno llega a ser exactamente cero.
- **Con L1**, los coeficientes se van «apagando» uno a uno. Con C muy pequeño sobreviven solo unas pocas características: L1 realiza **selección automática de variables**.
- **En el panel derecho** se ve que con regularización excesiva (C muy pequeño) el modelo es demasiado simple y su exactitud cae: es **subajuste**. Existe una zona intermedia con buena exactitud y pocas variables.

```
In [8]:
l1 = LogisticRegression(C=0.05, l1_ratio=1, solver="liblinear").fit(X_train_s, y_train)
```

```
Out[8]:
L1 con C = 0.05 → 6 de 30 características  |  exactitud test 0.9790
worst radius            1.473
mean concave points     0.856
worst concave points    0.468
worst texture           0.388
worst symmetry          0.078
worst smoothness        0.001
```

Con solo **6 de las 30 características** el modelo alcanza 97.9 % de exactitud en prueba. Este modelo es mucho más fácil de explicar a un médico: depende del tamaño, la concavidad, la textura y la simetría del tumor. Además, L1 resolvió parcialmente la multicolinealidad: de las tres medidas de tamaño correlacionadas se quedó solo con `worst radius`.

> **ADVERTENCIA · ¿Cómo elegir C?**
>
> El valor de C es un **hiperparámetro** y debe elegirse con **validación cruzada** (`LogisticRegressionCV` o `GridSearchCV`), nunca observando el conjunto de prueba. Si elegimos C mirando el desempeño en prueba, ese conjunto deja de ser una estimación honesta del rendimiento futuro.

10 Preprocesamiento y trampas comunes

10.1 Escalado de características

El escalado afecta a la regresión logística por dos vías distintas:

1. **Convergencia.** Cuando las características tienen escalas muy distintas, la superficie de costo se convierte en un valle muy alargado y estrecho. El gradiente apunta casi perpendicular al fondo del valle y el algoritmo avanza en zigzag, necesitando muchísimas iteraciones. Con características estandarizadas el valle es más «redondo» y el camino al mínimo es directo.
2. **Regularización justa.** La penalización trata a todos los pesos por igual. Una variable medida en mm^2 (valores de miles) necesita un peso diminuto y apenas es penalizada; otra en escala 0–1 necesita un peso grande y es penalizada fuertemente. Estandarizar pone a todas en igualdad de condiciones.

```
Out[8]:
Sin escalar:  2657 iteraciones (L-BFGS) | exactitud test 0.9441
Escalado   :    20 iteraciones (L-BFGS) | exactitud test 0.9650
```

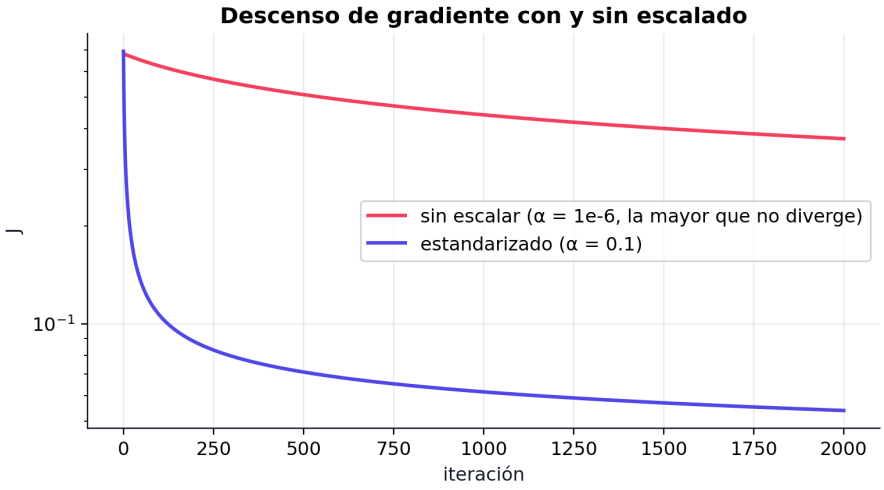


Figura 9. *Descenso de gradiente con y sin escalado. Sin escalar, la mayor tasa de aprendizaje que no diverge (10^{-6}) produce un avance casi nulo.*

Incluso el sofisticado L-BFGS necesita más de 100 veces más iteraciones sin escalar, y además obtiene peor exactitud. Con nuestro descenso de gradiente simple el efecto es dramático: la columna `mean area` obliga a usar un α diminuto para no diverger, y con ese α las demás características apenas aprenden.

10.2 Variables categóricas

La regresión logística solo opera con números. Las variables nominales (ciudad, tipo de sangre, sistema operativo) se transforman con **one-hot encoding**: una columna binaria por categoría (`OneHotEncoder`

de scikit-learn o `pd.get_dummies`). Cuando no hay regularización conviene eliminar una de las columnas (`drop="first"`) para evitar colinealidad perfecta.

> ADVERTENCIA
>
> **Nunca** codifiques una variable nominal como rojo = 1, verde = 2, azul = 3. El modelo interpretaría un orden (azul > verde > rojo) y una distancia (azul está «el doble de lejos» de rojo que verde) que no existen.

10.3 Separabilidad perfecta

Si existe un hiperplano que separa **perfectamente** las clases del entrenamiento, ocurre algo curioso: multiplicar **w** por una constante mayor que 1 no cambia la frontera, pero hace la sigmoide más empinada y acerca todas las probabilidades a 0 o 1, reduciendo el costo. Sin regularización, el costo siempre puede mejorarse agrandando los pesos, de modo que **los pesos crecen sin límite** y el algoritmo nunca converge. Las probabilidades resultantes son extremas y sobreconfiadas.

Out[8]:

```
sin regularización,     10 iteraciones → w =   1.420
sin regularización,    100 iteraciones → w =   2.851
sin regularización,   1000 iteraciones → w =   4.884
sin regularización,  10000 iteraciones → w =   7.138
sin regularización,  50000 iteraciones → w =   8.742
con L2 (λ = 1)    ,  50000 iteraciones → w =   1.136   <- se estabiliza
```

En este ejemplo con ocho puntos separables, el peso no deja de crecer mientras haya iteraciones. Con regularización L2 se estabiliza en un valor razonable. Es una de las razones por las que scikit-learn regulariza **por defecto**.

11 Evaluación del clasificador

La **exactitud** (*accuracy*), el porcentaje de aciertos, es la métrica más intuitiva, pero puede ser muy engañosa. Imagina un sistema de detección de fraude donde solo el 1 % de las transacciones son fraudulentas: un «modelo» que responda siempre *no es fraude* tendría 99 % de exactitud y sería completamente inútil. Necesitamos métricas que distingan **qué tipo** de errores comete el modelo.

11.1 Matriz de confusión

La matriz de confusión cruza la clase real con la predicha y separa los cuatro resultados posibles:

	Predicho 0 (negativo)	Predicho 1 (positivo)
Real 0	VN — verdadero negativo	FP — falso positivo (error tipo I)
Real 1	FN — falso negativo (error tipo II)	VP — verdadero positivo

A partir de estos cuatro números se definen las métricas principales:

Métrica	Fórmula	Pregunta que responde
Exactitud	(VP + VN) / total	¿Qué fracción acierto en total?
Precisión	VP / (VP + FP)	De los que marqué como positivos, ¿cuántos lo eran?
Recall (sensibilidad)	VP / (VP + FN)	De los positivos reales, ¿cuántos detecté?
Especificidad	VN / (VN + FP)	De los negativos reales, ¿cuántos identifiqué?
F1	2·P·R / (P + R)	Media armónica de precisión y recall

En el diagnóstico de cáncer, un **falso negativo** (decir «benigno» ante un tumor maligno) es mucho más grave que un falso positivo, que solo implica estudios adicionales. Por eso en este problema prestaremos especial atención al **recall**.

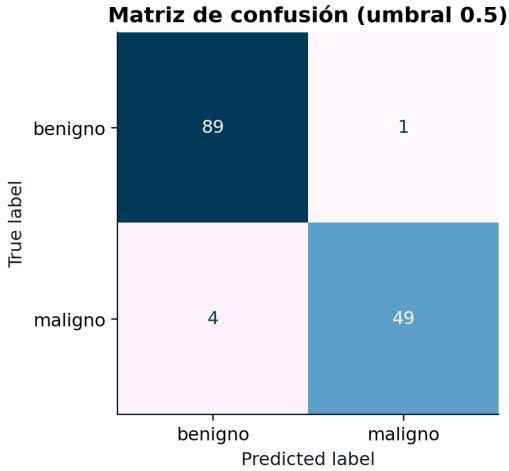


Figura 10. *Matriz de confusión del modelo con C = 1 sobre el conjunto de prueba, con umbral 0.5.*

Out[8]:

	precision	recall	f1-score	support
benigno	0.957	0.989	0.973	90
maligno	0.980	0.925	0.951	53
accuracy			0.965	143

De 53 tumores malignos el modelo detecta 49 (recall 92.5 %) y se le escapan 4. De los 50 casos que marcó como malignos, 49 lo eran (precisión 98 %). La pregunta natural es: ¿podemos detectar esos 4 tumores perdidos? La respuesta está en el umbral.

11.2 Curvas ROC y Precisión-Recall

Todas las métricas anteriores dependen del umbral $\tau = 0.5$. Las curvas evalúan el modelo **para todos los umbrales posibles** a la vez:

- **Curva ROC.** Grafica la tasa de verdaderos positivos (recall) frente a la tasa de falsos positivos (1 − especificidad). Su resumen es el **AUC** (área bajo la curva), que tiene una interpretación elegante: es la probabilidad de que el modelo asigne mayor puntaje a un positivo elegido al azar que a un negativo elegido al azar. AUC = 0.5 equivale a lanzar una moneda; AUC = 1 es un clasificador perfecto.
- **Curva Precisión-Recall.** Grafica precisión frente a recall. Es más informativa cuando la clase positiva es **rara**, porque no se deja «inflar» por la gran cantidad de verdaderos negativos. Su resumen es la **precisión promedio** (AP); la línea base es la prevalencia de la clase positiva.

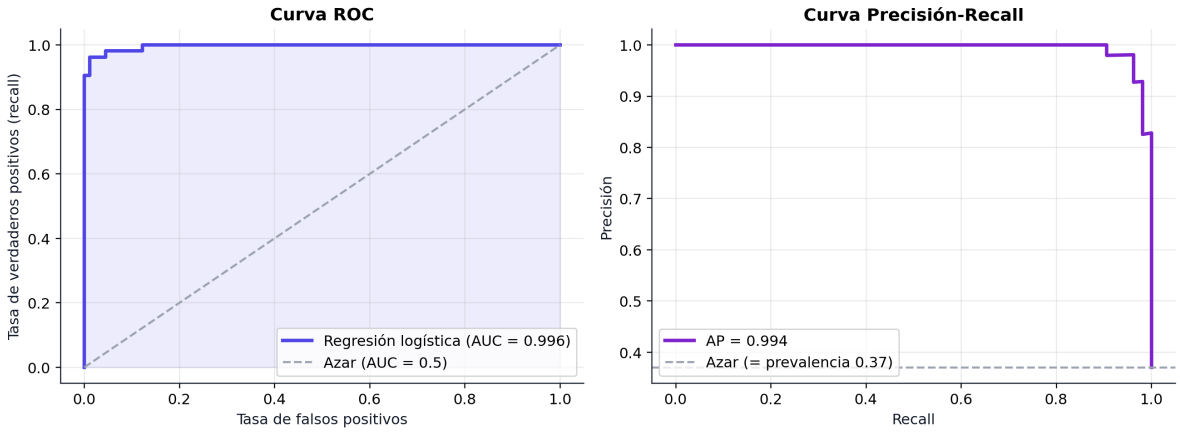


Figura 11. *Curva ROC (AUC = 0.996) y curva Precisión-Recall (AP = 0.994) del modelo sobre el conjunto de prueba.*

11.3 Elección del umbral

El umbral 0.5 **no es sagrado**. Es una decisión que depende del costo relativo de cada tipo de error en el contexto del problema:

Contexto	Error más costoso	Estrategia de umbral
Tamizaje médico	Falso negativo (no detectar la enfermedad)	Umbral **bajo** → más recall
Filtro de spam	Falso positivo (perder un correo importante)	Umbral **alto** → más precisión
Detección de fraude	Depende del costo de revisar vs. la pérdida	Minimizar el costo esperado

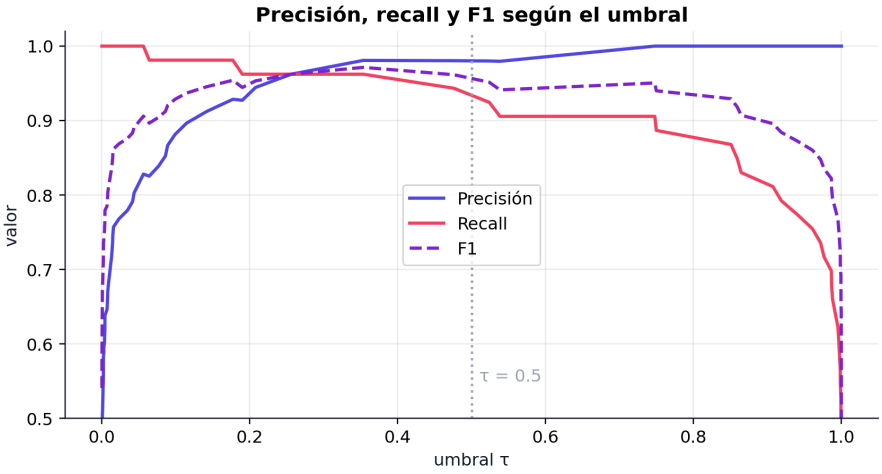


Figura 12. *Precisión, recall y F1 en función del umbral τ.*

Out[8]:

umbral	precisión	recall	F1	FN	FP
0.1	0.897	0.981	0.937	1	6
0.2	0.944	0.962	0.953	2	3
0.3	0.981	0.962	0.971	2	1
0.5	0.980	0.925	0.951	4	1
0.7	1.000	0.906	0.950	5	0
0.9	1.000	0.811	0.896	10	0

Al bajar el umbral de 0.5 a 0.3, los falsos negativos pasan de 4 a 2 sin aumentar prácticamente los falsos positivos: una mejora clara para un problema médico. Con $\tau = 0.1$ solo se escapa un tumor, a costa de 6 falsas alarmas. La elección final corresponde al experto del dominio, que conoce el costo real de cada error.

> **ADVERTENCIA**
>
> Igual que C, el umbral debe elegirse con un conjunto de **validación**, no con el de prueba. Aquí usamos el conjunto de prueba solo con fines ilustrativos.

11.4 Clases desbalanceadas

Cuando una clase es muy minoritaria, el modelo tiende a ignorarla, porque equivocarse con ella apenas afecta al costo promedio. Las estrategias principales son:

1. **Ponderar las clases** con `class_weight="balanced"`: cada ejemplo pesa en el costo de forma inversamente proporcional a la frecuencia de su clase, de modo que equivocarse con la minoritaria «duele» más.
2. **Remuestrear**: sobremuestrear la clase minoritaria (por ejemplo con SMOTE, de la librería `imbalanced-learn`) o submuestrear la mayoritaria.
3. **Ajustar el umbral** y evaluar con métricas adecuadas (recall, F1, AP), **nunca** solo con exactitud.

Simulamos un problema con solo 5.3 % de positivos:

```
Out[8]:
                modelo  exactitud  precisión   recall      F1
          sin ponderar      0.966      0.960    0.375   0.539
 class_weight=balanced      0.793      0.181    0.812   0.295
           «siempre 0»      0.947          –    0.000   0.000
```

El modelo sin ponderar parece excelente (96.6 % de exactitud), apenas mejor que el clasificador trivial «siempre 0» (94.7 %), pero **se le escapan casi dos de cada tres positivos**. Con `class_weight="balanced"` el recall sube de 37.5 % a 81.2 %, a cambio de muchas falsas alarmas: la precisión cae al 18 % y la exactitud baja al 79 %.

> **IDEA CLAVE**
>
> Ninguna de las dos opciones es «la correcta» en abstracto. Si los positivos son fraudes o enfermedades graves, normalmente preferiremos el segundo modelo; si cada falsa alarma es muy cara, el primero. **Lo que importa es el costo de cada error, no la exactitud.**

12 Clasificación multiclase

Muchos problemas tienen más de dos clases: reconocer dígitos (10 clases), clasificar especies de flores (3) o identificar el tipo de ataque en una red. Cuando $y \in \{1, 2, \ldots, K\}$ con $K > 2$, existen dos estrategias principales.

12.1 One-vs-Rest (OvR)

Se entrenan **K clasificadores binarios independientes**. El k-ésimo aprende a distinguir «clase k» frente a «todas las demás». Para predecir, se calcula la probabilidad que da cada uno y se elige la clase cuyo clasificador está más seguro. Es una estrategia simple que funciona con cualquier clasificador binario, pero tiene limitaciones: las K probabilidades no suman 1 de forma natural y cada modelo se entrena sin conocer a los demás.

12.2 Regresión softmax (multinomial)

Es la generalización natural de la regresión logística. Se entrena **un único modelo** con un vector de pesos $\mathbf{w}_k$ y un sesgo b_k **por cada clase**. Primero se calcula un logit por clase, $z_k = \mathbf{w}_k^T \mathbf{x} + b_k$, y luego se convierten en probabilidades con la función **softmax**:

$$P(y = k \mid \mathbf{x}) = \text{softmax}(\mathbf{z})_k = e^{z_k} / \sum_j e^{z_j} \tag{12.1}$$

La exponencial garantiza que todos los valores sean positivos, y la división por la suma garantiza que **sumen 1**. La clase con mayor logit obtiene la mayor probabilidad, pero las demás no quedan en cero: el modelo expresa su incertidumbre.

EJEMPLO · Cálculo de softmax

Para los logits $z = (2.0, 1.0, 0.1)$: $e^{2} = 7.389$, $e^{1} = 2.718$, $e^{0.1} = 1.105$, cuya suma es 11.212.

softmax(z) = (7.389/11.212, 2.718/11.212, 1.105/11.212) = **(0.659, 0.242, 0.099)**, que suman 1.

Con $K = 2$ la softmax se reduce exactamente a la sigmoide aplicada a la diferencia de logits, $\sigma(z_1 - z_0)$: la regresión logística binaria es un caso particular de la softmax. La función de costo es la **entropía cruzada categórica**, donde $y^{(i)}$ se escribe en codificación one-hot (un 1 en la posición de la clase real y ceros en las demás):

$$J = -(1/m) \cdot \sum_i \sum_k y_k^{(i)} \cdot \ln \hat{p}_k^{(i)}$$
$$\partial J / \partial W = (1/m) \cdot X^T (\hat{P} - Y) \tag{12.2}$$

Como solo hay un 1 en cada vector one-hot, la doble suma se reduce a $-\ln$ de la probabilidad asignada a la clase correcta. Y el gradiente conserva exactamente la misma forma que en el caso binario.

> NOTA · **Estabilidad numérica de softmax**
>
> En la práctica se resta el máximo de los logits antes de exponenciar: softmax(**z**) = softmax(**z** − máx(**z**)). El resultado es idéntico (el factor e^(−máx) se cancela entre numerador y denominador) pero se evita el desbordamiento con logits grandes.

12.3 Ejemplo con el dataset Iris

Usamos el clásico dataset Iris (150 flores de 3 especies) con solo dos características, el largo y el ancho del pétalo, para poder visualizar las fronteras. En scikit-learn ≥ 1.7 `LogisticRegression` usa softmax automáticamente cuando hay más de dos clases; para OvR se envuelve el modelo con `OneVsRestClassifier`.

In [9]:
```
softmax_clf = make_pipeline(StandardScaler(), LogisticRegression()).fit(Xi_tr, yi_tr)
ovr_clf = make_pipeline(StandardScaler(), OneVsRestClassifier(LogisticRegression())).fit(Xi_tr,
yi_tr)
```

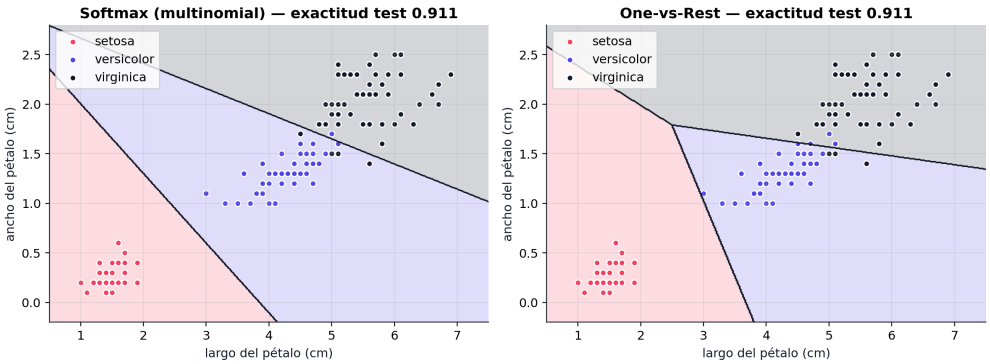


Figura 13. *Regiones de decisión para tres especies de Iris con softmax (izquierda) y One-vs-Rest (derecha). Ambos alcanzan 91.1 % de exactitud en prueba.*

Out[9]:
```
Flor con pétalo 4.9 × 1.6 cm:
  softmax: setosa: 0.004, versicolor: 0.573, virginica: 0.422
  OvR    : setosa: 0.008, versicolor: 0.480, virginica: 0.513
```

Ambos métodos producen fronteras lineales por tramos y la misma exactitud. Sin embargo, para una flor ubicada en la zona de duda entre *versicolor* y *virginica*, los dos modelos **discrepan**: softmax se inclina por versicolor y OvR por virginica. Es un recordatorio de que las probabilidades cerca de la frontera son inciertas. En la práctica se suele preferir softmax por ser un único modelo coherente; además, es exactamente la **capa de salida** de las redes neuronales para clasificación multiclase.

13 Síntesis y cierre

13.1 Resumen de fórmulas

Concepto	Fórmula
Modelo	$\hat{p} = \sigma(\mathbf{w}^T\mathbf{x} + b)$
Sigmoide	$\sigma(z) = 1 / (1 + e^{-z})$
Derivada	$\sigma'(z) = \sigma(z)\cdot(1 - \sigma(z))$
Logit	$\ln(p / (1 - p)) = \mathbf{w}^T\mathbf{x} + b$
Frontera ($\tau = 0.5$)	$\mathbf{w}^T\mathbf{x} + b = 0$
Costo con L2	$J = -(1/m)\cdot\sum[y\cdot\ln \hat{p} + (1 - y)\cdot\ln(1 - \hat{p})] + (\lambda/2m)\cdot\|\|\mathbf{w}\|\|^2$
Gradiente	$\partial J/\partial \mathbf{w} = (1/m)\cdot X^T(\hat{p} - y) + (\lambda/m)\cdot\mathbf{w}$
Actualización	$\mathbf{w} \leftarrow \mathbf{w} - \alpha\cdot\partial J/\partial \mathbf{w}$
Odds ratio	$e^{(w_i)}$
Softmax	$\hat{p}_k = e^{(z_k)} / \sum_j e^{(z_j)}$

13.2 Ventajas y limitaciones

Ventajas	Limitaciones
Simple, rápida de entrenar y de usar en producción	Frontera lineal: requiere ingeniería de características para patrones no lineales
Coeficientes interpretables mediante odds ratios	La interpretación se degrada con multicolinealidad
Produce probabilidades, no solo etiquetas	Pesos inestables ante separabilidad perfecta sin regularización
Costo convexo: óptimo global garantizado	Requiere escalado y codificación cuidadosos
Excelente modelo de referencia (baseline)	Suele ser superada por modelos no lineales en datos complejos

13.3 Flujo de trabajo recomendado

1. Explorar los datos: tamaño, proporción de clases, escalas y correlaciones.
2. Dividir en entrenamiento y prueba con `stratify` **antes** de cualquier transformación.
3. Preprocesar dentro de un `Pipeline`: codificación one-hot y estandarización.
4. Entrenar `LogisticRegression` eligiendo C (y el tipo de penalización) por validación cruzada.
5. Evaluar con la matriz de confusión, precisión, recall, F1, ROC-AUC y AP, no solo con exactitud.
6. Elegir el umbral según el costo de cada error, usando un conjunto de validación.

7. Interpretar los coeficientes con cautela y comunicar los resultados en términos del problema.

13.4 Puente hacia las redes neuronales

Una regresión logística **es una neurona artificial**: recibe las entradas **x**, las multiplica por los pesos **w**, suma el sesgo b y aplica una función de activación (la sigmoide). Una red neuronal no es más que muchas de estas unidades organizadas en capas, donde la salida de unas es la entrada de otras. Su algoritmo de entrenamiento, *backpropagation*, es la misma regla de la cadena que aplicamos en la sección 6, repetida capa por capa. Todo lo aprendido aquí —log-loss, softmax, descenso de gradiente, tasa de aprendizaje, mini-batches y regularización— se reutiliza directamente al estudiar redes neuronales.

14 Ejercicios propuestos

Los ejercicios de cálculo se resuelven a mano; los de programación se apoyan en el cuaderno Jupyter que acompaña a estos apuntes.

Ejercicios de cálculo y comprensión

1. Calcula $\sigma(z)$ para $z \in \{-2, -1, 0, 1, 2\}$ y verifica numéricamente la propiedad $\sigma(-z) = 1 - \sigma(z)$.
2. Un modelo tiene $w = (1.2, -0.8)$ y $b = 0.5$. Calcula $\hat{p}$ para $x = (1, 2)$ y $x = (-1, 0.5)$. Escribe la ecuación de la frontera de decisión en la forma $x_2 = m \cdot x_1 + c$.
3. Un modelo para predecir aprobación tiene, para la variable *horas de estudio*, un coeficiente $w = 0.4$. Calcula e interpreta el odds ratio. ¿Por cuánto se multiplican los odds al estudiar 5 horas más?
4. Demuestra que con $K = 2$ la función softmax aplicada a (z_0, z_1) da como probabilidad de la clase 1 exactamente $\sigma(z_1 - z_0)$.

Ejercicios de programación

1. Entrena `RegresionLogisticaGD` con el dataset de horas de estudio de la sección 1. ¿A partir de cuántas horas la probabilidad de aprobar supera 0.5? ¿Y 0.9?
2. Modifica la clase para que acepte regularización L1 (pista: el subgradiente de $|w|$ es $\text{sign}(w)$). Compara los coeficientes obtenidos con los de scikit-learn usando `l1_ratio=1`.
3. Usa `LogisticRegressionCV` con 5 *folds* sobre Breast Cancer para elegir C. Reporta el C óptimo, la exactitud y el AUC en el conjunto de prueba.
4. Supón que un falso negativo cuesta 10 veces más que un falso positivo. Escribe una función que encuentre el umbral que minimiza el costo total y compáralo con 0.5.
5. Extiende la implementación a K clases (pesos $W \in \mathbb{R}^{n \times K}$, softmax, entropía cruzada categórica). Entrénala sobre las 4 características de Iris y compara con scikit-learn.
6. Aplica el flujo de trabajo completo al dataset *Pima Indians Diabetes* o *Titanic*, incluyendo la codificación one-hot de las variables categóricas, y redacta un breve informe de resultados.